



Python Nivel intermedio - EPC VVG04.

FUNCIONES PARA MODULARIDAD.

UNIDAD 1.

1B. Ejercicio. Lambda, decoradores, generadores

ALUMNO: HURTADO LÓPEZ PEDRO.

PROFESOR: ORTEGA FIGUEROA EFRÉN
ALEJANDRO.

Domingo 08 de marzo del 2026.

Introducción.

Este ejercicio práctico está diseñado para consolidar y aplicar los conceptos fundamentales de las funciones para modularidad que has estudiado en la Unidad 1. Las funciones vistas son las siguientes:

- Funciones lambda
- Funciones de orden superior
- Decoradores
- Generadores.

Ejercicio 1. Planteamiento del problema.

Sistema de Monitoreo Ambiental Inteligente: Procesador de Alertas de Temperatura.

Una empresa de monitoreo ambiental requiere un módulo en Python para procesar datos de temperatura, recogidos por sensores en diferentes ciudades. Los requisitos para la construcción del modelo serán:

1. Generador de Datos. Crea una función generadora *leer_temperaturas()*:

```
# Lista de Temperaturas --> Datos Sensores
datos_temp = [("CDMX", 26),
               ("MORELIA", 24),
               ("QUERETARO", 33),
               ("MONTERREY", 36),
               ("CANCUN", 35),
               ("LEON", 28),
               ("GUADALAJARA", 31),
               ("CELAYA", 21),
               ("SAN LUIS", 30)]

#Funcion generadora
def leer_temperaturas():
    for ciudad, temp in datos_temp:
        yield ciudad, temp
```

```
Lista de Datos de Sensores - Ciudades y Temperaturas [('CDMX', 26), ('MORELIA', 24), ('QUERETARO', 33), ('MONTERREY', 36), ('CANCUN', 35), ('LEON', 28),
('GUADALAJARA', 31), ('CELAYA', 21), ('SAN LUIS', 30)]
```

2. Filtrado con filter() y lambda. Filtra los registros que tengan temperaturas *mayores o iguales a 30°C*:

```
# Funcion para filtrar
def may_30():
    may_30c = lambda x: x [1] >= 30 # Ciudades mayores o iguales a 30°C
    return list(filter(may_30c, leer_temperaturas()))
```

```
Las Ciudades calientes son: [('QUERETARO', 33), ('MONTERREY', 36), ('CANCUN', 35), ('GUADALAJARA', 31), ('SAN LUIS', 30)]
```

3. Transformación con map() y lambda. Convierte cada tupla a un string con el formato → *"Alerta de calor en [Ciudad]: [Temperatura]°C"*:

```
def procesador_alertas():
    ciudades_calientes = may_30() # Asignación ciudades calientes
    # print(ciudades_calientes)

    # Transformacion con map() y lambda
    alerta_calor = lambda ciudad_caliente: ("Alerta de calor en", ciudad_caliente[0], "con temperatura de", ciudad_caliente[1])
    Alerta = list(map(alerta_calor, ciudades_calientes))
    # print(Alerta)
```

```
[('Alerta de calor en', 'QUERETARO', 'con temperatura de', 33), ('Alerta de calor en', 'MONTERREY', 'con temperatura de', 36), ('Alerta de calor en', 'CANCUN', 'con temperatura de', 35), ('Alerta de calor en', 'GUADALAJARA', 'con temperatura de', 31), ('Alerta de calor en', 'SAN LUIS', 'con temperatura de', 30)]
```

4. Ordenamiento con sorted() y key=lambda. Ordena los registros filtrados por temperatura en *orden descendente*:

```
# Ordenamiento los datos filtrados, orden descendente
ordenar_por_temp = lambda temp: temp[1]
ciudad_ord = list(sorted(ciudades_calientes, key=ordenar_por_temp, reverse=True))
print("\nLas ciudades con alerta son", ciudad_ord)
```

```
Las ciudades con alerta son [('MONTERREY', 36), ('CANCUN', 35), ('QUERETARO', 33), ('GUADALAJARA', 31), ('SAN LUIS', 30)]
```

5. Resumen con reduce(). Calcula el *promedio de temperaturas mayores a 30°C*:

```
#Promedio de temperatura de alerta
tam= len(ciudad_ord)
# print(tam)
valores = list (map(lambda x: x[1], ciudad_ord))
# print(valores)
sum = reduce(lambda x, y: x + y, valores)
# print(sum)
prom = sum / tam
print(f"Temperatura promedio de alertas: {prom:.2f}°C")
```

```
Temperatura promedio de alertas: 33.00°C
```

6. Decorador personalizado. Crea un decorador llamado @auditar_funcion:
 - a. Imprima el nombre de la función que se está ejecutando.
 - b. Cuente cuántas veces ha sido llamada.
 - c. Opcional: muestre la duración de ejecución (puede reutilizar el decorador @medir_tiempo visto en los recursos básicos).

```
# Decorador personalizado
def auditar_funcion(func):
    cont = 0 # Iniciar contador

    def decorador(*args, **kwargs):
        nonlocal cont # Definir que no es una variable local
        cont += 1 # Incrementa el contador
        inicio = time.time() # Mide tiempo de inicio

        print("Se ejecuto la funcion: ", func.__name__) # func.__name__ para poner nombre de la función
        print("Se ejecuto la funcion: ", cont)

        fin = time.time() # Mide tiempo final
        print(f"Tiempo de ejecucion: {fin - inicio:.7f} segundos")

        return func(*args, **kwargs)
    return decorador
```

```
Se ejecuto la funcion: procesador_alertas
Se ejecuto la funcion: 1
Tiempo de ejecucion: 0.0001729 segundos
```

7. Salida final esperada.
 - a. Imprimir la lista ordenada de alertas.
 - b. Imprimir el promedio de temperaturas en formato: *"Temperatura promedio de alertas: 33.5°C"*

SALIDA EN VS Code.

```
PS C:\Users\hurta\Documents\2026\curso-pyinter> & C:\Users\hurta\AppData\Local\Programs\Python\Python313\python.exe c:/Users/hurta/PycharmProjects/curso-pyinter/Ejercicio_U1_LambdaDecoradoresGeneradores/Ejercicio_U1.py
Se ejecuto la funcion: procesador_alertas
Se ejecuto la funcion: 1
Tiempo de ejecucion: 0.0002310 segundos

Las ciudades con alerta son [('MONTERREY', 36), ('CANCUN', 35), ('QUERETARO', 33), ('GUADALAJARA', 31), ('SAN LUIS', 30)]
Temperatura promedio de alertas: 33.00°C
PS C:\Users\hurta\Documents\2026\curso-pyinter> & C:\Users\hurta\AppData\Local\Programs\Python\Python313\python.exe c:/Users/hurta/PycharmProjects/curso-pyinter/Ejercicio_U1_LambdaDecoradoresGeneradores/Ejercicio_U1.py
Se ejecuto la funcion: procesador_alertas
Se ejecuto la funcion: 1
Tiempo de ejecucion: 0.0003767 segundos

Las ciudades con alerta son [('MONTERREY', 36), ('CANCUN', 35), ('QUERETARO', 33), ('GUADALAJARA', 31), ('SAN LUIS', 30)]
Temperatura promedio de alertas: 33.00°C
```

SALIDA EN COLAB.

```
*** Se ejecuto la funcion: procesador_alertas
Se ejecuto la funcion: 1
Tiempo de ejecucion: 0.0000823 segundos

Las ciudades con alerta son [('MONTERREY', 36), ('CANCUN', 35), ('QUERETARO', 33), ('GUADALAJARA', 31), ('SAN LUIS', 30)]
Temperatura promedio de alertas: 33.00°C
```

Conclusiones.

- En conclusión, esta ejercicio nos permitió integrar conocimientos teóricos, reforzando la comprensión de las funciones de orden superior como: `filter()`, `map()`, `sorted()`, y `reduce()`, así como la función `lambda`, que nos sirve para realizar las operaciones simples en el ejercicio como filtrar, ordenar o para calcular el promedio de temperaturas. También podemos observar como el `@decorador` nos ayuda a mostrar la función que se está ejecutando, cuantas veces y su duración, sin alterar la función original del código, solo nos ayuda a agregar funcionales adicionales sin modificar el código original.